



Software Architecture 2024[2026]

Lecture 3, part 2/2

Software architecture defined

Dimitri Van Landuyt

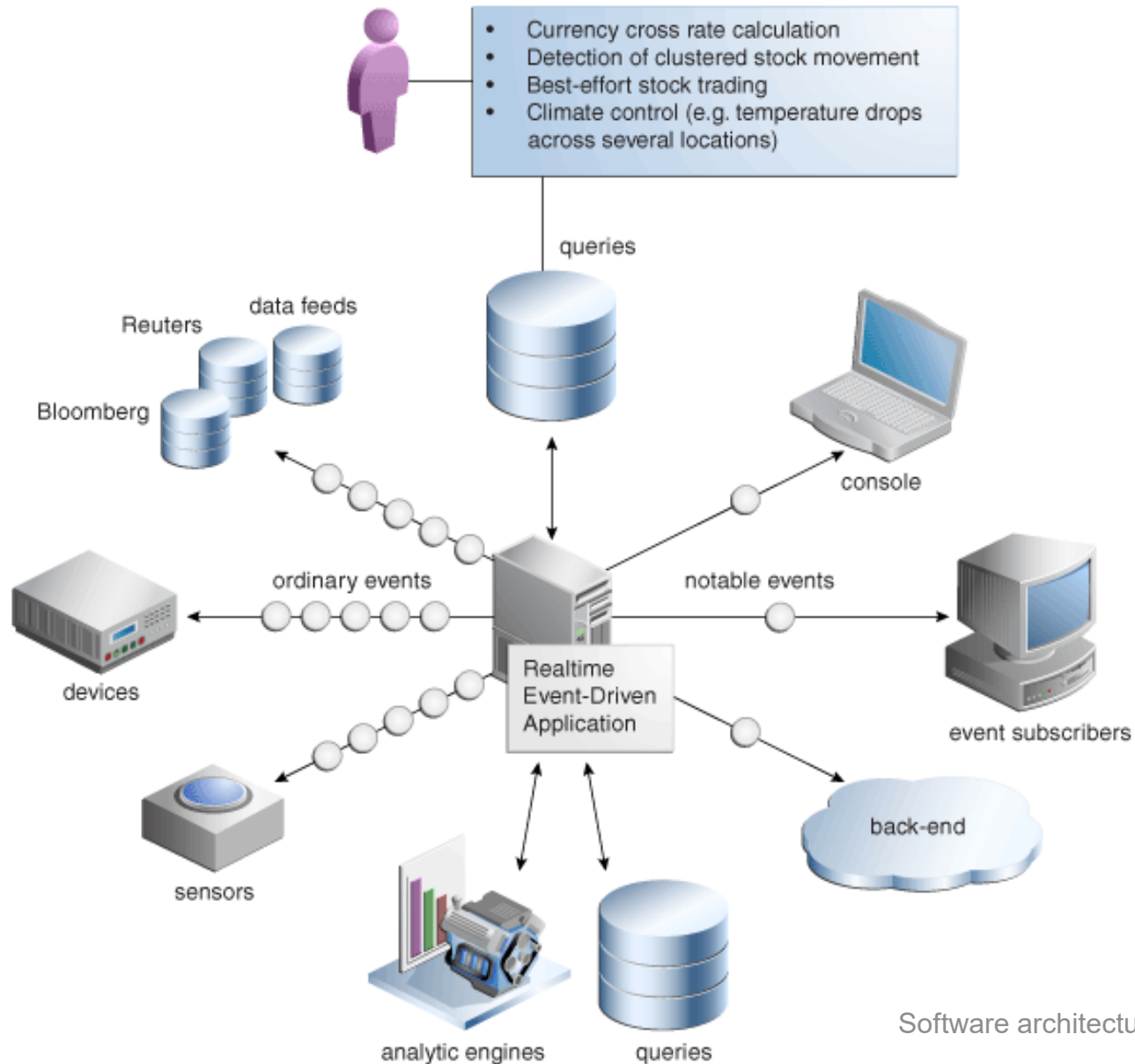
Wouter Joosen

Laurens Sion

What is a software architecture?

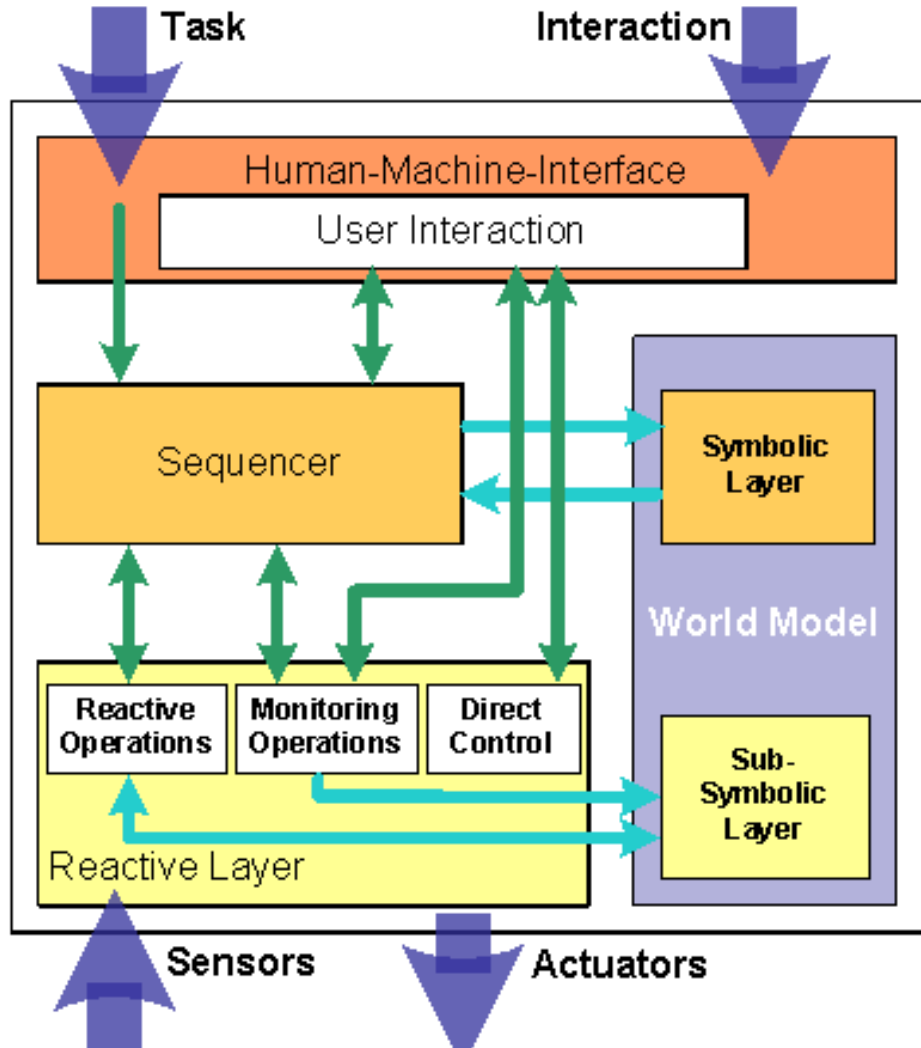
What is a software architecture?

This?



What is a software architecture?

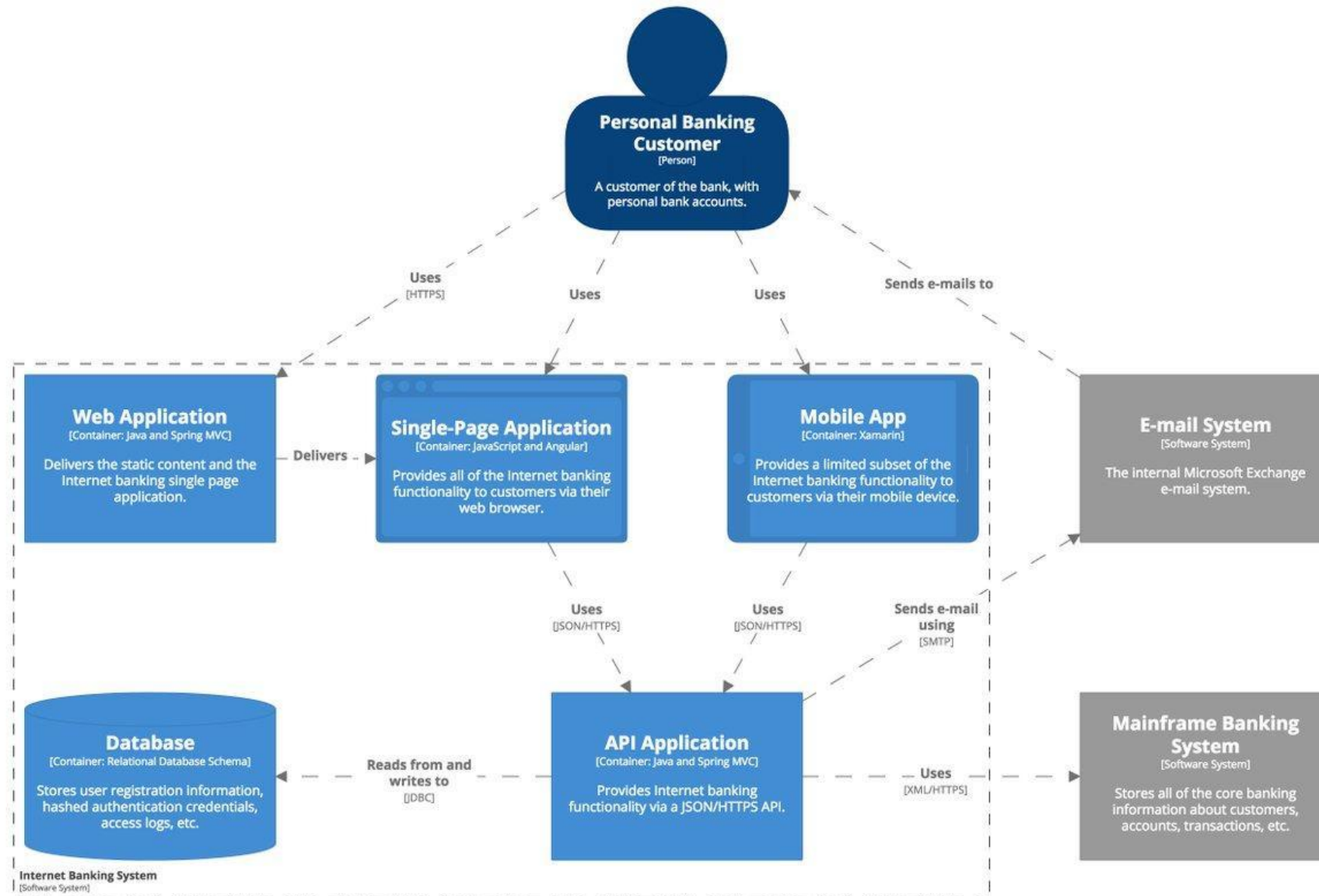
This?



Source:
 Hybrid multi-layer control architecture for semi-autonomous service robots with verified task execution.
https://en.wikipedia.org/wiki/Care-Providing_Robot_FRIEND

What is a software architecture?

This?



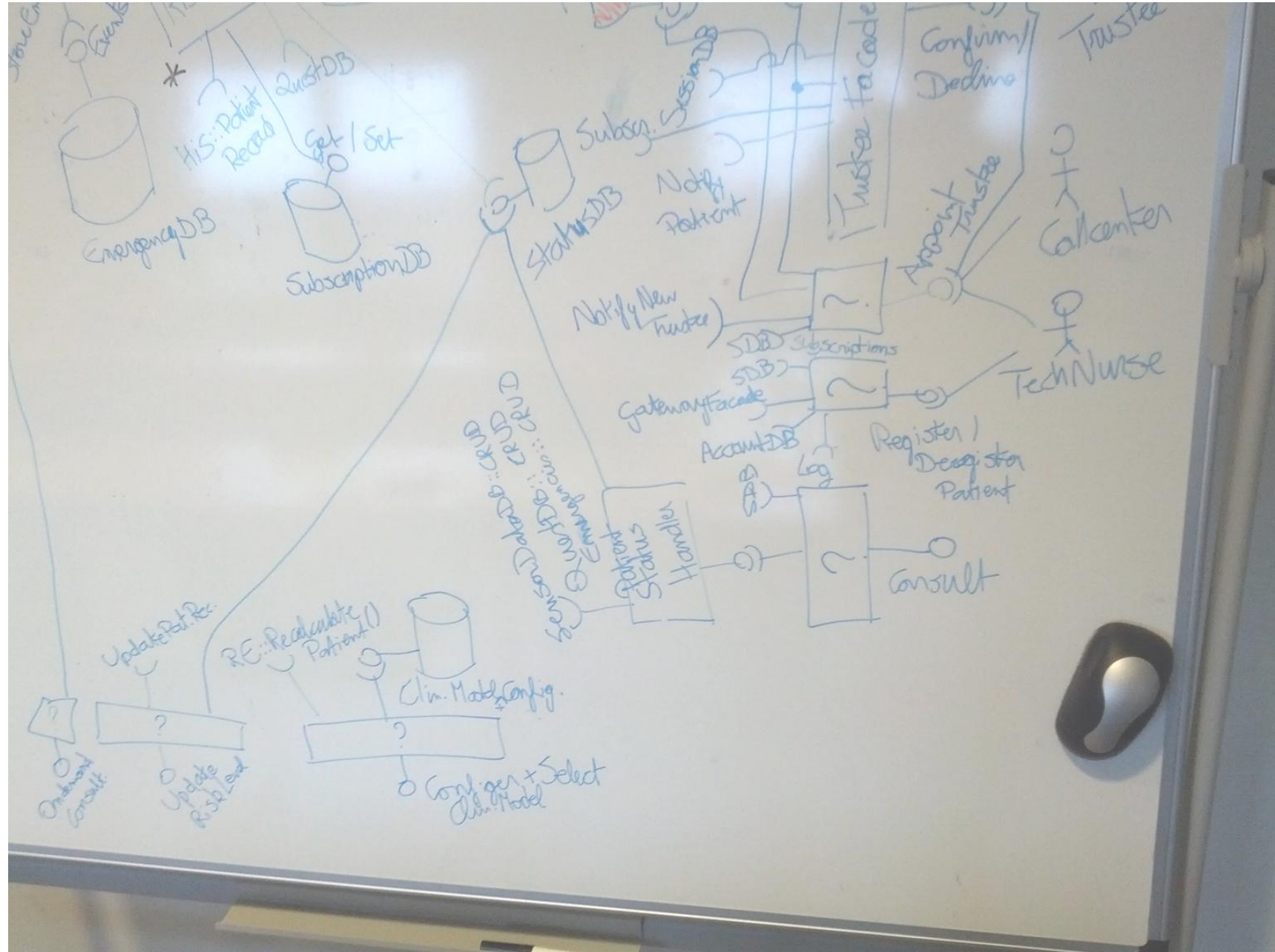
Container diagram for Internet Banking System

The container diagram for the Internet Banking System.

Last modified: Wednesday 02 May 2018 13:46 UTC

What is a software architecture?

This?



What is a software architecture?

A diagram?

A style?

Patterns?

Diagram

Layers

Icon

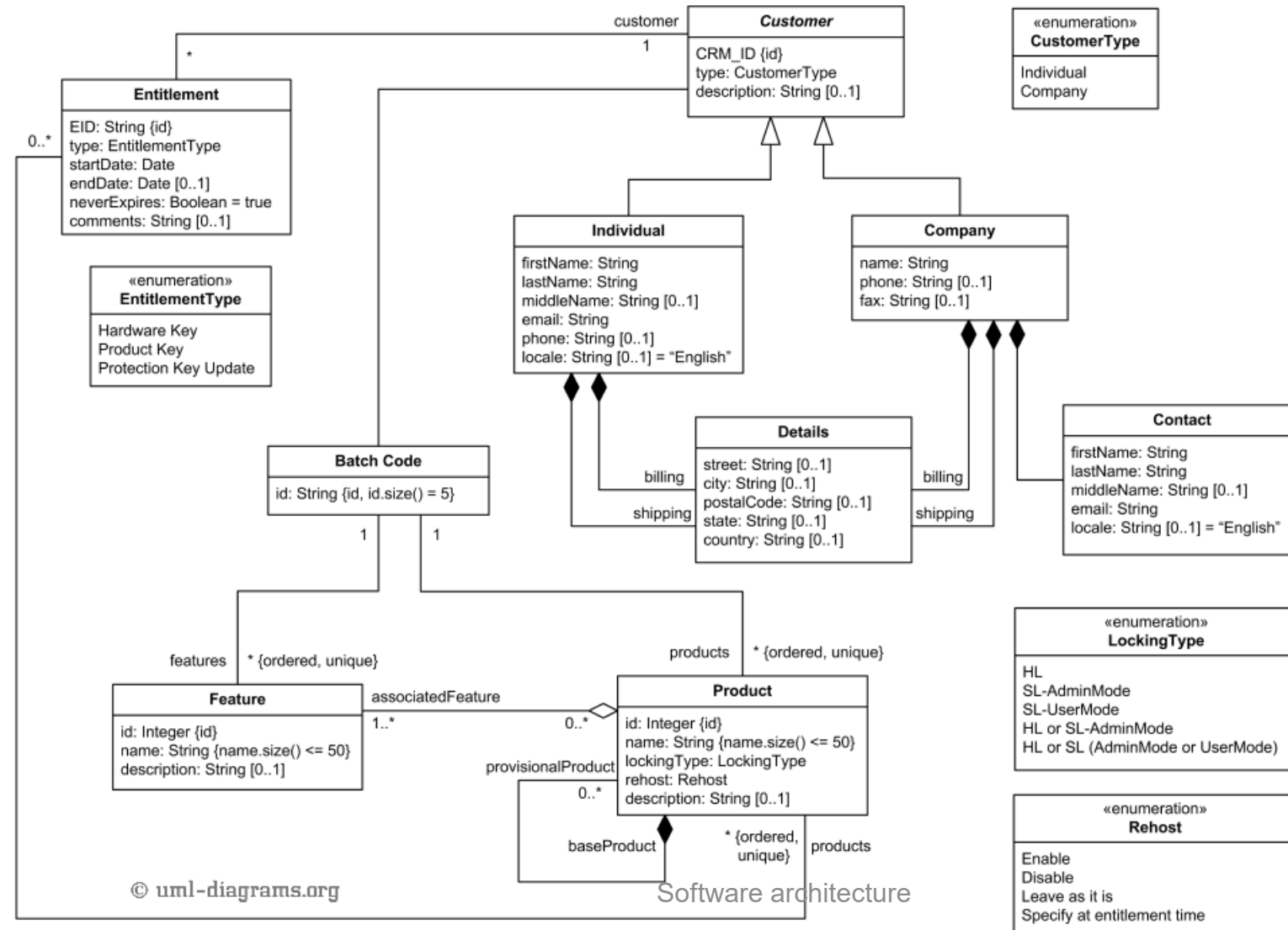
SOA

Simple

What is a software architecture?

UML diagram?

SW model?



What is architecture?



Definition

ANSI/IEEE Standard 1471-2000

Architecture is the fundamental organization of a system, embodied in

- its components,*
- their relationships to each other and to the environment and*
- the principles guiding its design and evolution*

Recommended Practice for Architectural Description of Software-Intensive Systems

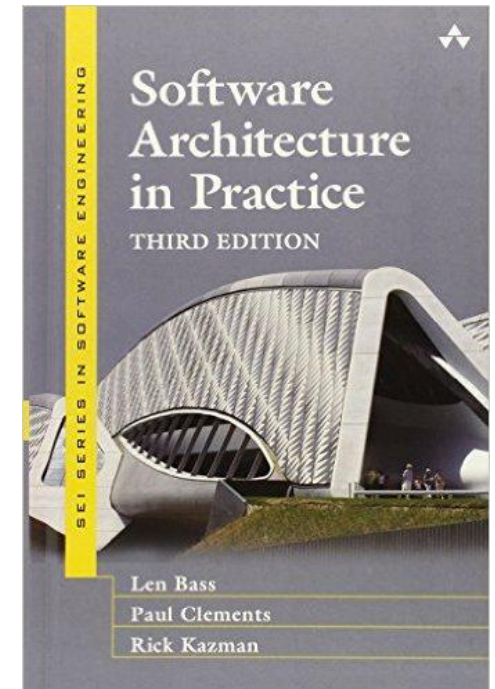
Definition

Bass, Clements and Kazman

*“The software architecture of a system is the **set of structures** needed to **reason** about the system, which comprise software **elements**, **relations** among them, and **properties** of both.”*

Bass, L. Clements, P. and Kazman, R. 2013 *Software Architecture in Practice*. 3rd edition.

Software architecture



Definition

Perry and Wolf 1992

Architecture = {elements, form, rationale}
 what how why

- › **Elements**: e.g., processing, data ...
- › **Form**: composition and distribution
 - ›› configuration, topology
 - ›› relationship among elements and with the environment
- › **Rationale**: designer's intent, assumptions, external constraints, decisions

Definition

Taylor, Medvidovic, Dashofy

- › A software system's architecture is the set of **principal design decisions** made about the system
- › **Principal** = of higher importance, as opposed to detailed design
- › **Design decisions** related to
 - › **System structure** “The modules should be organized like this”
 - › **Functional behavior** “Data processing, storage and visualization are performed in strict sequence”
 - › **Interaction** “Communication occurs via event notifications”
 - › **Nonfunctional properties** “The system dependability will be ensured by replicated processing modules”
 - › **Implementation** “Client modules are web-based”

Note well

- › **Every** system has a software architecture
 - › The more *software-intensive*, the more complicated its software architecture...
- › Architecture can exist **independently of its description or specification**
 - › Hence the need for architecture **documentation**
 - › Hence the need for architecture **reconstruction**

Our understanding

(SOFTWARE-INTENSIVE) SYSTEM

SOFTWARE ARCHITECTURE

shapes

(SOFTWARE-INTENSIVE) SYSTEM

SOFTWARE ARCHITECTURE

Structure: elements, and (de)composition (relationships)

Principles: tactics, patterns, styles, design principles

Rationale: decisions, assumptions

=> In function of Software Quality

shapes



(SOFTWARE-INTENSIVE) SYSTEM

SOFTWARE ARCHITECTURE

Structure: elements, and (de)composition (relationships)

Principles: tactics, patterns, styles, design principles

Rationale: decisions, assumptions

=> In function of **Software Quality**

shapes



(SOFTWARE-INTENSIVE) SYSTEM

*conveys
information
about*



**Architecture
description**

intangible

SOFTWARE ARCHITECTURE

Structure: elements, and (de)composition

Principles: tactics, patterns, styles, design principles

Rationale: decisions, assumptions

=> In function of **Software Quality**

shapes

(SOFTWARE-INTENSIVE) SYSTEM

tangible

*conveys
information
about*

tangible

**Architecture
description**

Let's get our feet wet

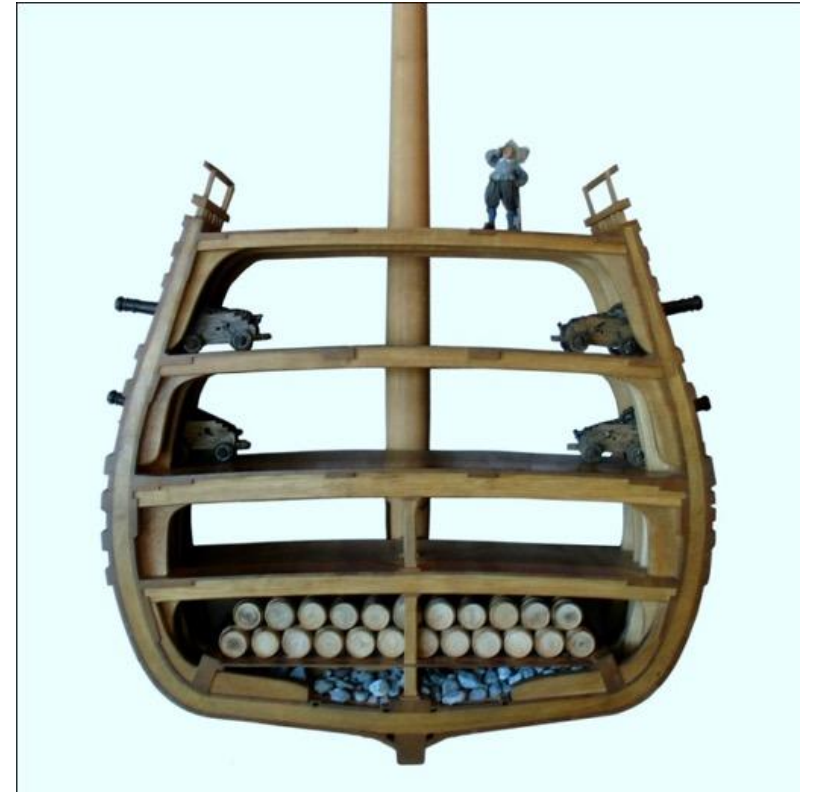
Out of the box, and into the water

- › Gustavus Adolphus,
king of Sweden
was at war with Poland
 - › Thirty Years' War (1618–1648)
- › In 1626, he asked the shipbuilder
Henrik Hybertsson (a Dutchman)
to design and build the **Vasa** warship



Out of the box, and into the water

- » **Innovative design**
 - »» much **bigger/taller** and
 - »» with **two decks of cannons**



- » On 10 August 1628, the Vasa **sank** just outside the Stockholm harbor,
 - » after sailing **less than a nautical mile** (ca 2 km) into its maiden voyage

[http://en.wikipedia.org/wiki/Vasa_\(ship\)](http://en.wikipedia.org/wiki/Vasa_(ship))

Software architecture



Oops!

Something went wrong. The requested page is not available in vasamuseet.se

[Back to front page vasamuseet.se](#)

If you typed the address by hand, check your spelling and try again.

If you clicked on a link to vasamuseet.se and got up this page, please write to support@vasamuseet.se . Including any which link you clicked on.



Why did it sink?

- › **Requirements creep**/changing needs
 - ›› The King frequently changed his orders for ships to be built
 - ›› Requirements for the armaments were changed repeatedly
- › **Lack of technical specifications**/project plan
 - ›› No specifications for the Vasa's modified keel were prepared
- › **Lack of scientific methods**
 - ›› There were no known methods for calculating factors such as stability, stiffness, and sailing characteristics of ships
 - ›› Different teams used different measuring systems
- › **Lead architect**
 - ›› Henrik Hybertsson became ill and died in 1627
- › **Ignoring the obvious**/mendacity
 - ›› A stability test conducted before launching the Vasa showed that the ship was not seaworthy

Architecture and software quality

Functionality, Qualities and Architecture

- › Functionality is **largely independent** of structure
- › Software architecture **constrains the allocation of functionality to structure** when other **quality attributes** are important

What drives the creation of an architecture?

- › Quality attributes **determine** the software architecture
 - › **Quality attributes shape the distribution architecture**
 - › **Quality attributes drive the decomposition**
 - › **Quality attributes dictate the deployment architecture**
- › Functionality is assigned to each component
- › Given the **WHAT** to achieve, the value of an architecture is in **HOW** the goals are achieved

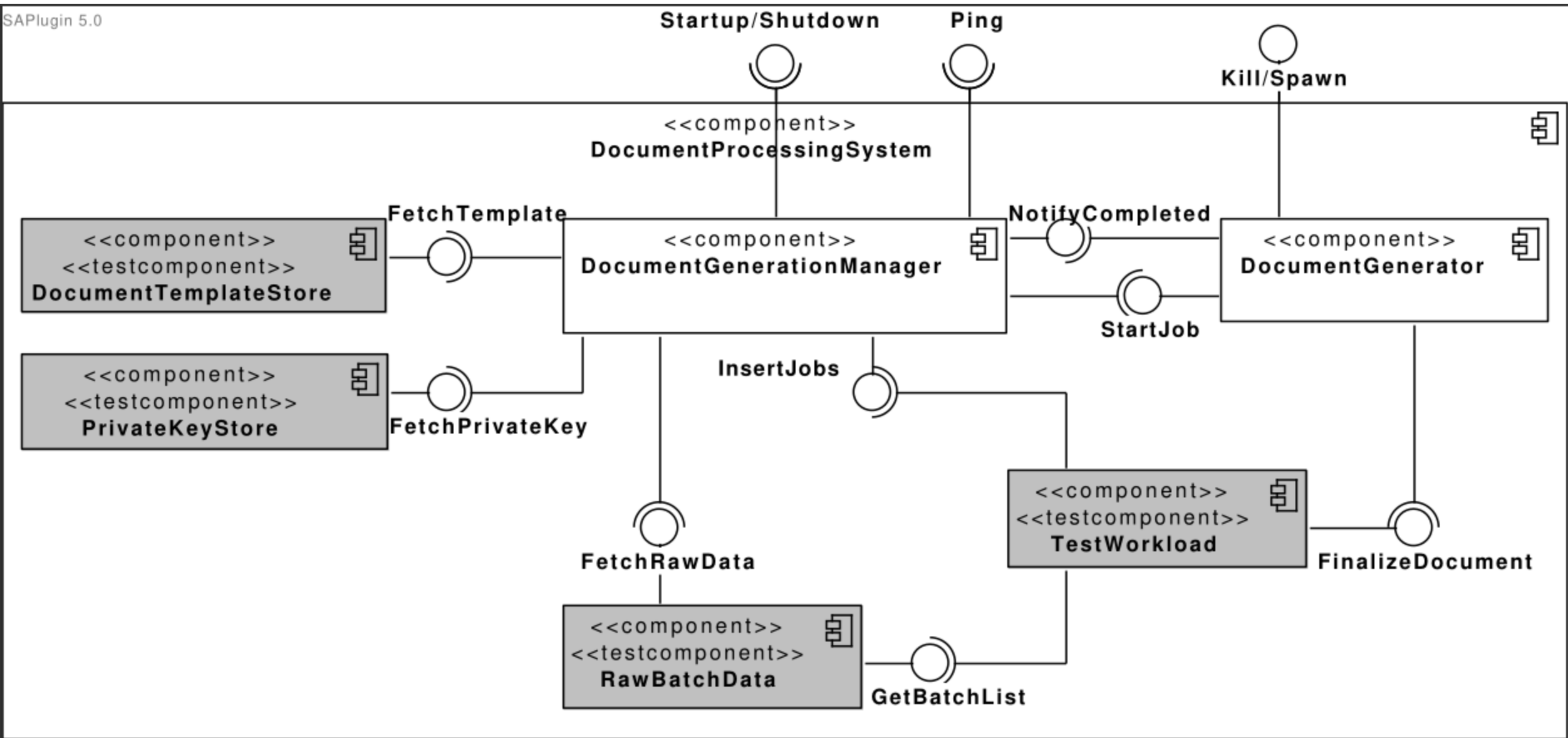
Therefore...

We focus primarily on **software quality**

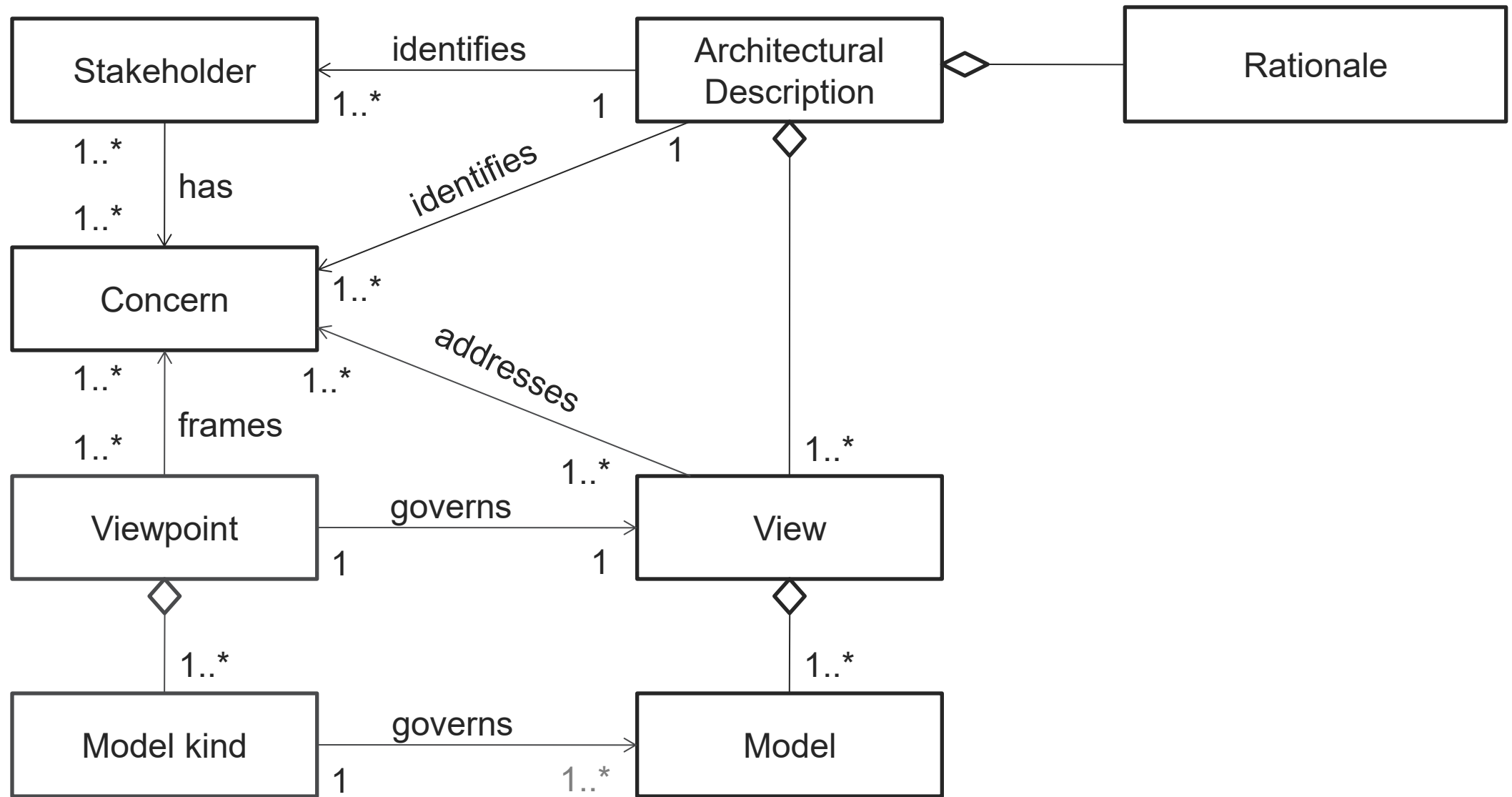
- › In **preparation of part 2** (architecture extension), you will study **software quality** aspects in an initial architecture,
 - › To identify **design decisions** already made
 - › To assess **trade-offs decisions** already made

- › During **part 2** (architecture extension), you will extend this to support a number of additional quality requirements, focusing on
 - › the **coherence** of **design decisions** with the expected **qualities**
 - › which design **alternatives** are considered
 - › which **trade-offs** have been made

Initial DocProc architecture

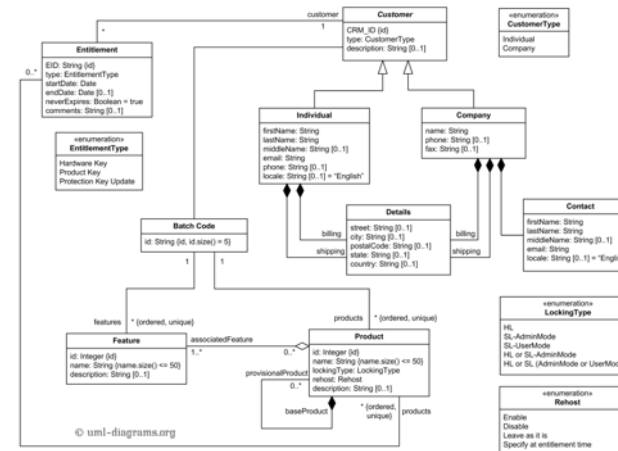
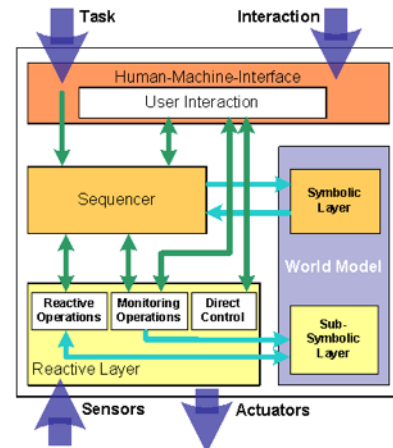
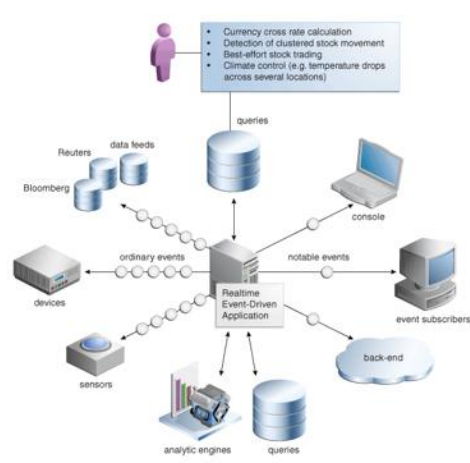


A preview on architecture description



On documenting the architecture

- › No single structure can claim to be “**the**” architecture
 - ›› But many structures will convey architectural information
- › In most cases, systems **need to be** described from many stand points
 - ›› Different non-quality requirements affect different aspects of the system
 - ›› More on this later



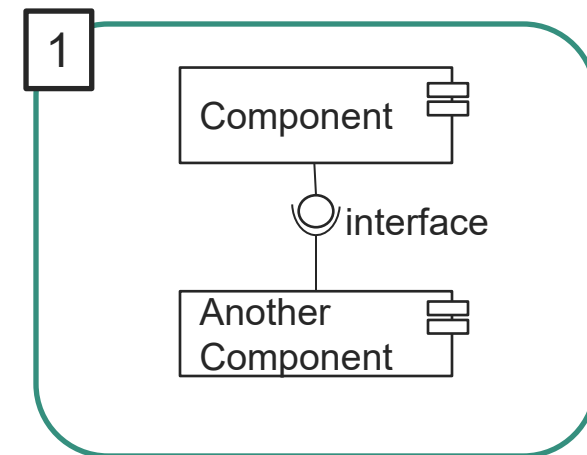
Architectural views

- › An architectural view is a simplified description (**abstraction**) of a system
 - ›› From a particular **perspective** (**stakeholders** and **concerns**) ~ viewpoint
 - ›› Omitting entities that are not relevant to this perspective
- › Contains **models** addressing all the concerns

In this course

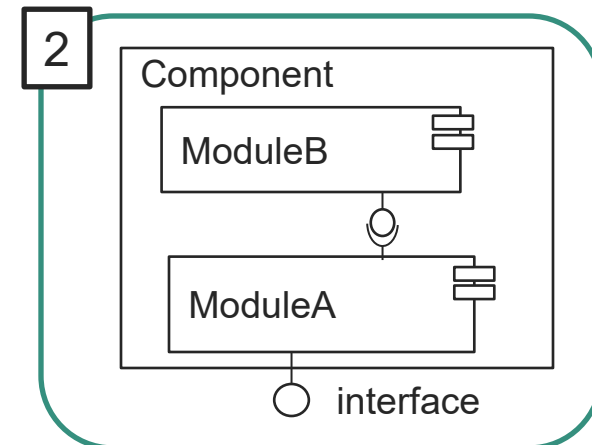
1. Client-server view

- » How the entire system is structured as a set of **elements that have runtime behavior and engage in interactions**:
- » **COMPONENTS**



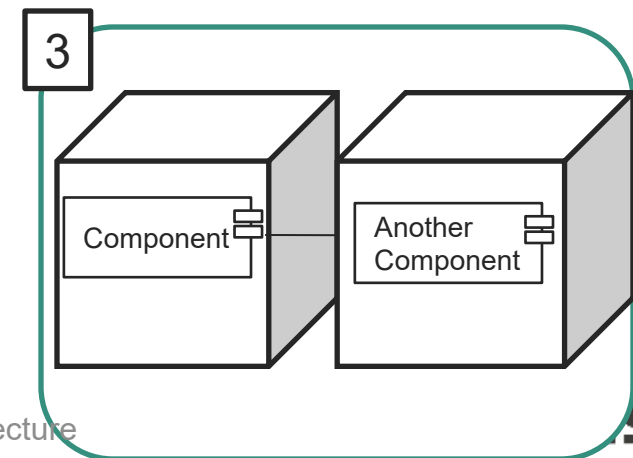
2. Decomposition view

- » How the system is decomposed into **units of implementation which guide the development process**:
- » **MODULES**



3. Deployment view

- » Specifies **how the software relates to non-software structures** (hardware, network topology):
- » **NODES**



4. Process view

- » **BEHAVIOR**: further refinement of the behavior and interactions of
- » **COMPONENTS**

Final thoughts

No silver bullet

- › Software architecture provides **foundations** to the realization of qualities
- › But achieving quality attributes must also be considered in **implementation, deployment, ...**
- › Satisfactory results are a matter of getting the **big picture** (architecture) as well as **the details** (implementation) right

No silver bullet

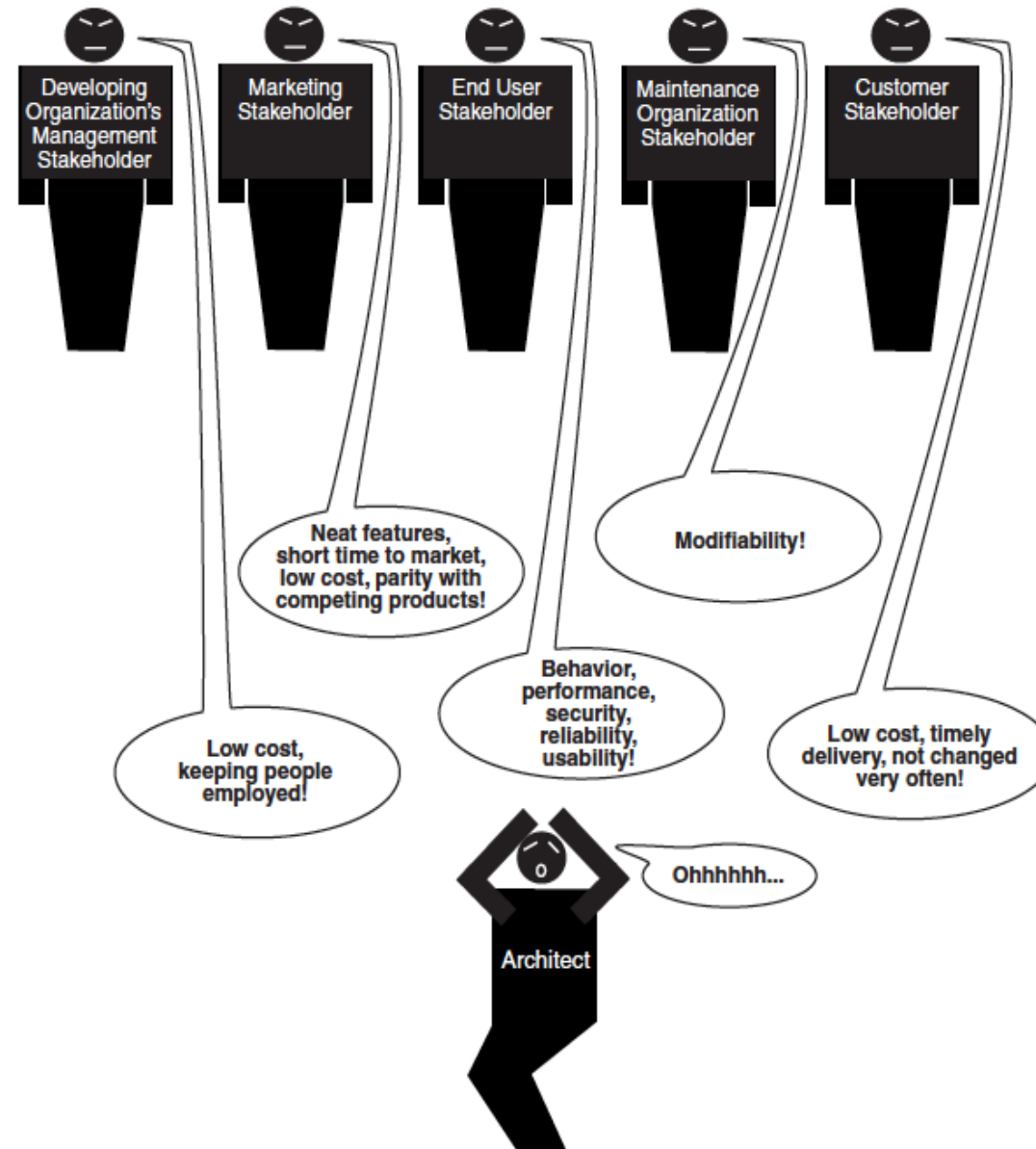
Examples

- › **Modifiability** is determined by
 - ›› How functionality is divided (architectural)
 - ›› Design principles within a module (non-architectural)

- › **Performance** requires
 - ›› Addressing bottlenecks (architectural)
 - ›› Smart coding practices, algorithmic optimization, and compilers (non-architectural)

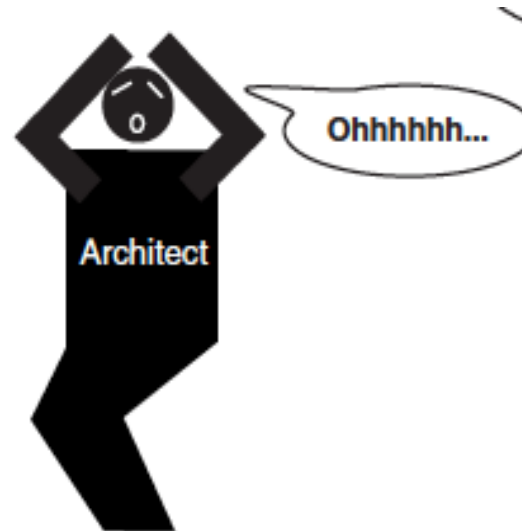
- › Security, availability, ...

Task of the architect



Task of the architect

› $\text{utility}(\text{asr}_1, \text{asr}_2, \text{asr}_3, \text{asr}_4 \dots \text{asr}_{15})$



Task of the architect

› $\text{utility}(\text{asr}_1, \text{asr}_2, \text{asr}_3, \text{asr}_4 \dots \text{asr}_{15})$

Let's assume **3 possible solutions per ASR**

Solving 1 requirement: 3 options

Solving 2 requirements $\sim 3^2$ options = 9

Solving 3 requirements $\sim 3^3 = 27$

...

Solving 15 requirements $\sim 3^{15} = 14,348,907$

Task of the architect

› $\text{utility}(\text{asr}_1, \text{asr}_2, \text{asr}_3, \text{asr}_4 \dots \text{asr}_{15})$

Let's assume **3 possible solutions per ASR**

Solving 1 requirement: 3 options

Solving 2 requirements $\sim 3^2$ options = 9

Solving 3 requirements $\sim 3^3 = 27$

...

Solving 15 requirements $\sim 3^{15} = 14,348,907$

Ohhhhhh...



Task of the architect

› $\text{utility}(\text{asr}_1, \text{asr}_2, \text{asr}_3, \text{asr}_4 \dots \text{asr}_{15})$

Let's assume **3 possible solutions per ASR**
and **solutions can be stacked**

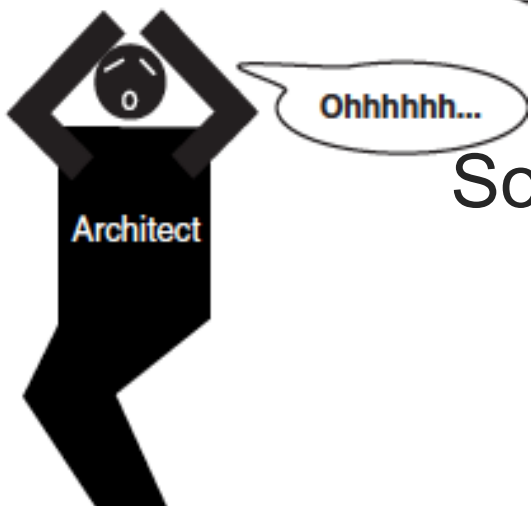
Solving 1 requirement $\sim 3!$ options = 6

Solving 2 requirements $\sim (3!)^2$ options = 36

Solving 3 requirements $\sim (3!)^3 = 216$

...

Solving 15 requirements $\sim 3^{15} = 470,184,984,576$



Task of the architect

The architect must

- ›› **understand** and **set priorities**,
- ›› make **trade-off decisions** among conflicting requirements and constraints
- ›› produce a **system abstraction** that is *complete, correct, understandable, buildable* within project constraints

*“The **problems** we face have an **amazing ability to scale**. Put five problems in a space, and they can all interact to make things 25 times worse. [...]*

Put five problem solvers in a room, however, and it is very unlikely that we will get even five times the problem-solving capability.” --

Tom Munnecke

Software architecture

Architecture vs. detailed design

Where is the line?

- › The terms “*architecture*”, “*design*” and “*implementation*” connote **varying degrees of abstraction** in the **continuum** between
 - › complete details (“implementation”),
 - › few details (“design”),
 - › and the highest form of abstraction (“architecture”).

Eden and Kazman, Architecture, Design, Implementation, ICSE 2003

Architecture vs. detailed design

Where is the line?

- › Architecture design **contains enough detail to explain and reason** about the *responsibilities** of elements (in light of the qualities)
 - › Sufficiently constrain further development activities
- › **Suppress** details that do not affect how they use, are used by, relate to, or interact with other elements (**abstraction**)
 - › E.g., internal state

* Remember GRASP?

Why is Software Architecture important?

Transferable abstraction of a system

- › **Blueprint specification** that sets the course of the development
- › Architecture enables more accurate **cost and schedule estimates**
- › Architecture enables early **evaluation** (e.g., ATAM, simulations, formal methods, ...)
- › The architecture makes it easier to reason about and manage **change** and **evolution**...
 - » 80% of the work is after initial deployment

Why is Software Architecture important?

Communication among stakeholders

- › The software architecture provides a **common language** in which different concerns can be expressed, negotiated and resolved
 - › at a level that is **intellectually manageable** even for large, complex systems
- › Architecture can be the basis for **training**

software architecture

Software architecture, definitions so far

- › Lect1: “*the bridge between requirements and design*”
- › Lect2: “*meeting or maximizing utility*”
- › Lect3: “*{elements, form, rationale}*” | “*the set of principal design decisions*” | “*the fundamental organization of a system, embodied in its components, their relationships [...], and the principles guiding its design and evolution*”

